

RAREBLOOD

Rare Blood Donor Alert Directory

Project Report

Full-Stack CRUD Web Application • HTML5 • CSS3 • JavaScript • Vite
Django • Django REST Framework • SQLite

Project Title	RareBlood – Rare Blood Donor Alert Directory
Student Name	Swetha S
Register Number	922525105059
Department	Electrical and Electronics Engineering (EEE)
College Name	VSB Engineering College
GitHub Repository	github.com/swetha745/rare-blood-donor-directory

OVERVIEW

Report Contents

2. Project Overview	15. CRUD Implementation
3. Problem Statement	16. Frontend–Backend Integration
4. Objectives	17. Validation & Error Handling
5. Requirement Analysis	18. Testing
6. Technology Stack	19. Security & Quality
7. System Architecture	20. Git & GitHub
8. Project Structure	21. Installation & Execution
9. Database Design	22. Challenges & Solutions
10. ER Diagram	23. Future Enhancements
11. Frontend Development	24. Learning Outcomes
12. User Interface	25. Conclusion
13. Backend Development	26. References
14. REST API Documentation	27. GitHub Repository

INTRODUCTION

2. Project Overview

RareBlood – Rare Blood Donor Alert Directory is a full-stack CRUD web application that maintains a centralised, searchable directory of blood donors, with particular attention to rare blood groups. It is developed as a college project using a vanilla HTML, CSS and JavaScript frontend served through Vite, and a Django REST Framework backend storing data in an SQLite database.

What problem it solves

During a medical emergency, locating a compatible donor is a time-critical task. Donor details are usually scattered across phone contacts, paper registers and informal messages, with no structured way to filter by blood group or location and no reliable indication of who is currently available. RareBlood replaces that scattered approach with a single validated directory that can be queried instantly by blood group and location.

Main features

- **Donor Directory** — lists every registered donor with name, blood group, location, phone and availability.
- **Find a Donor** — filters the directory by blood group and location.
- **Register Donor** — validated form that creates a new donor record.
- **Edit Donor** — updates an existing donor record, most often to change availability.
- **Delete Donor** — removes a donor record permanently.
- **Blood Requests** — records and lists requests with hospital, units required and urgency.

What this report covers

The report documents the problem and objectives, the requirement analysis, the technology stack and system architecture, the database and ER design, the frontend and backend implementation, the REST API, the CRUD data flow, validation and error handling, the test results, installation steps, the challenges encountered, future enhancements and the learning outcomes of the project.

THE PROBLEM

3. Problem Statement

Blood banks and hospitals cannot always supply a required blood group at short notice, and families are frequently asked to arrange a donor themselves. The search that follows is almost entirely manual.

Difficulty in finding blood donors

The pool of willing donors for a given blood group in a given city is small, and for rare groups such as AB–, B– or O– it may be only a handful of people. Those people are not recorded anywhere that the person searching can reach.

Problems with manual searching

- Phone calls and forwarded messages are slow and duplicate effort across many people.
- There is no way to filter contacts by blood group or by location.
- Records kept on paper or in chat groups become outdated and cannot be corrected centrally.
- There is no indication of whether a donor is currently available to donate.
- The same donor is contacted repeatedly while other willing donors are never reached.

Need for a centralised donor directory

What is required is a single place where donor details are stored in a structured form, can be searched by the two attributes that actually matter during an emergency — blood group and location — and can be kept current by editing or deleting records as circumstances change.

The proposed RareBlood solution

RareBlood provides exactly that. Donor details are entered through a validated web form and stored in an SQLite database through a Django REST API. The directory can be filtered by blood group and location, each record can be edited or deleted, and a separate section records blood requests with the hospital, the units required and an urgency flag.

Manual search		RareBlood
-----		-----
Phone calls, chat groups	--->	Structured donor records
No filtering	--->	Blood group + location search
Outdated contacts	--->	Editable, deletable records
Availability unknown	--->	Availability shown per donor

GOALS

4. Objectives

#	Objective	How it is met
1	Donor registration	A Register Donor form posts validated data to the REST API.
2	Donor searching	The directory is queried and re-rendered from API responses.
3	Blood-group and location filtering	Search inputs filter the donor list by both criteria.
4	Donor information management	Existing donor records can be edited and deleted.
5	Blood request management	A separate section stores and lists blood requests.
6	CRUD operations	Create, Read, Update and Delete are implemented for both entities.
7	Validation	Client-side checks in JavaScript and server-side checks in DRF serializers.
8	REST API integration	The frontend communicates with the backend using the Fetch API and JSON.
9	Database storage	All records are stored persistently in SQLite through the Django ORM.

ANALYSIS

5. Requirement Analysis

5.1 Target users

- Patients' families searching for a compatible donor during an emergency.
- Volunteers and donors who wish to list themselves in the directory.
- College NSS or blood donation camp coordinators maintaining a donor list.

5.2 Main entities

The system has two entities: **Donor**, representing a person willing to donate blood, and **BloodRequest**, representing a requirement raised for a patient.

5.3 Required fields

Entity	Required fields	Optional fields
Donor	name, blood_group, phone, location	email, available (defaults to true)
BloodRequest	name, blood_group, location, phone, hospital, units_required	urgent (defaults to false)

5.4 CRUD requirements

- Create — register a new donor; record a new blood request.
- Read — list all donors and all blood requests; retrieve a single record by id.

- Update — edit an existing donor record, including its availability status.
- Delete — permanently remove a donor record.

5.5 Search and filter requirements

- Filter the donor directory by blood group, selected from the eight standard groups.
- Filter the donor directory by location.
- Apply both filters together to narrow the result set.
- Display a clear message when no donor matches the criteria.

5.6 Validation requirements

- All required fields must be present before a record is accepted.
- Blood group must be one of the eight valid groups.
- Phone number must be numeric and of a valid length.
- Email, when supplied, must be in a valid format.
- Units required must be a positive number.
- Validation must be enforced on the server, not only in the browser.

STACK

6. Technology Stack

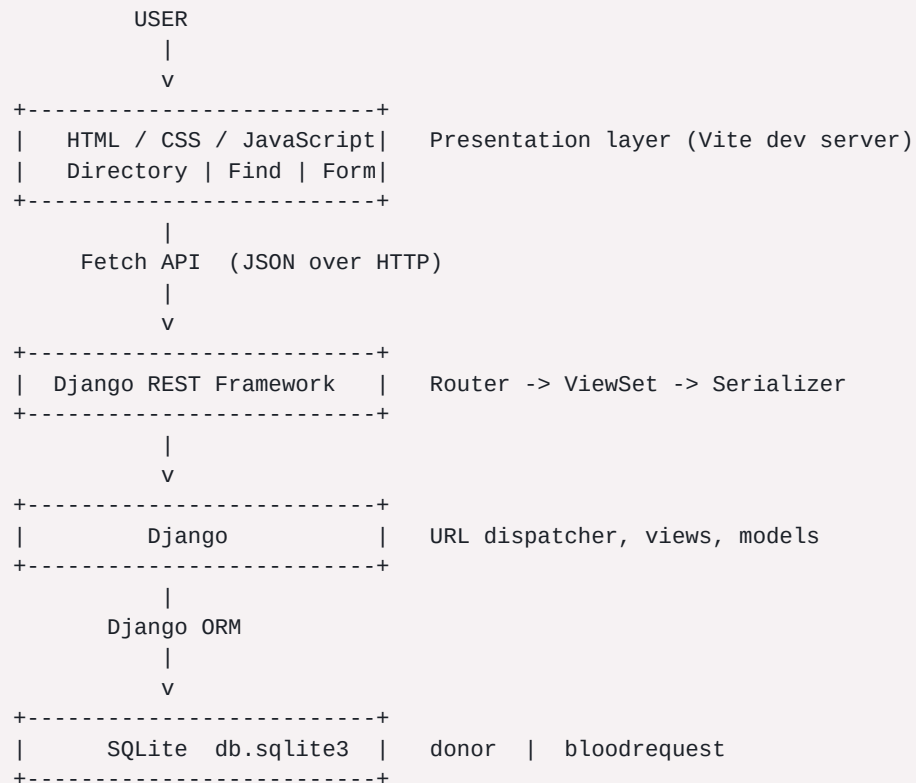
Layer	Technology	Purpose
Frontend	HTML5	Page structure and semantic markup for sections, forms and lists.
Frontend	CSS3	Styling, layout and responsive presentation of the interface.
Frontend	JavaScript (vanilla ES6)	DOM rendering, event handling, client-side validation and API calls.
Development Tool	Vite	Development server with hot reload and optimised production build.
Backend	Python	Server-side programming language.
Backend	Django 6.1.1	Web framework providing models, ORM, migrations, routing and admin.
REST API	Django REST Framework 3.18.1	Serializers, ViewSets and routers exposing the models as JSON endpoints.
Database	SQLite	File-based relational database storing donor and blood request records.
Communication	Fetch API, JSON	HTTP requests and responses between the frontend and the backend.
Version Control	Git, GitHub	Source code management, commit history and remote hosting.

The frontend is written in **plain JavaScript**. No frontend framework or component library is used; Vite serves only as the development server and build tool.

DESIGN

7. System Architecture

RareBlood follows a three-tier client–server architecture. The browser holds the presentation tier, Django holds the application tier, and SQLite holds the data tier. The tiers are fully decoupled: the browser never touches the database directly, and every read and write travels through the REST API.



How data travels between the layers

1. The user interacts with a form or a button in the browser.
2. JavaScript reads the input values and validates them on the client.
3. The Fetch API sends an HTTP request carrying a JSON body or query parameters.
4. Django's URL dispatcher routes the request to the DRF router, which selects the ViewSet action.
5. The serializer validates incoming data or converts outgoing model instances to JSON.
6. The Django ORM translates the operation into SQL and executes it against SQLite.
7. SQLite returns rows, which the ORM converts back into model instances.
8. DRF returns a JSON response with an appropriate HTTP status code.
9. JavaScript parses the JSON and updates the DOM without reloading the page.

CODEBASE

8. Project Structure

```
rare-blood-donor-directory/  
|  
+-- backend/  
|   +-- config/           Django project settings, root URL configuration  
|   +-- donors/           Django app: models, serializers, views, urls  
|   +-- db.sqlite3        SQLite database file  
|   +-- manage.py         Django management script  
|  
+-- public/               Static assets served by Vite  
+-- src/  
|   +-- main.js           All frontend logic: rendering, validation, Fetch calls  
|   +-- style.css         Stylesheet for the entire interface  
|  
+-- index.html            Single HTML entry point  
+-- package.json          Frontend dependencies and npm scripts  
+-- package-lock.json     Locked dependency versions  
+-- README.md             Project description and setup instructions  
+-- .gitignore            Excludes venv, node_modules and local files
```

The backend and frontend live in the same repository but remain independent of each other. `config/` holds the Django project configuration, while `donors/` is the application containing the models, serializers and ViewSets. On the frontend, `index.html` is the single entry point and all behaviour is implemented in `src/main.js`.

DATA MODEL

9. Database Design

The project uses **SQLite**, a lightweight file-based relational database that stores the entire database in a single file, `db.sqlite3`. It requires no separate server process and is bundled with Python, which makes it well suited to a single-user college project. All tables are created and modified through Django migrations rather than by writing SQL by hand.

9.1 Donor entity

Field	Type	Constraint	Description
id	BigAutoField	Primary key	Unique identifier generated automatically.
name	CharField	Required	Full name of the donor.
blood_group	CharField	Required, choices	One of A+, A−, B+, B−, AB+, AB−, O+, O−.
phone	CharField	Required	Contact number of the donor.
email	EmailField	Optional	Email address of the donor.
location	CharField	Required	City or area where the donor is based.
available	BooleanField	Default true	Whether the donor is currently available.
created_at	DateTimeField	Auto on create	Timestamp when the record was created.
updated_at	DateTimeField	Auto on save	Timestamp of the most recent update.

9.2 BloodRequest entity

Field	Type	Constraint	Description
id	BigAutoField	Primary key	Unique identifier generated automatically.
name	CharField	Required	Name of the patient or requester.
blood_group	CharField	Required, choices	Blood group required for the patient.
location	CharField	Required	City or area where the blood is needed.
phone	CharField	Required	Contact number of the requester.
hospital	CharField	Required	Hospital where the patient is admitted.
units_required	IntegerField	Required	Number of units of blood required.
urgent	BooleanField	Default false	Marks the request as urgent.
created_at	DateTimeField	Auto on create	Timestamp when the request was raised.
updated_at	DateTimeField	Auto on save	Timestamp of the most recent update.

SCHEMA

10. ER Diagram

The relationship between the two entities is summarised below. Since there is no foreign key between them, the diagram shows two independent tables rather than a connected schema.

+-----+				+-----+			
DONOR				BLOOD_REQUEST			
+-----+				+-----+			
PK	id	INTEGER		PK	id	INTEGER	
	name	VARCHAR			name	VARCHAR	
	blood_group	VARCHAR			blood_group	VARCHAR	
	phone	VARCHAR			location	VARCHAR	
	email	VARCHAR			phone	VARCHAR	
	location	VARCHAR			hospital	VARCHAR	
	available	BOOLEAN			units_required	INTEGER	
	created_at	DATETIME			urgent	BOOLEAN	
	updated_at	DATETIME			created_at	DATETIME	
+-----+				+-----+			

No foreign key exists between the two tables. A blood request is matched against donors at query time using blood_group and location, not through a stored relationship.

10.1 Explanation

Database entities. The database contains two entities, **Donor** and **BloodRequest**, stored as two independent tables.

Primary keys. Each entity has an id field created automatically by Django as a BigAutoField. It is the primary key and increments with every new record, guaranteeing that each row is uniquely identifiable.

Attributes. Donor carries name, blood_group, phone, email, location and available. BloodRequest carries name, blood_group, location, phone, hospital, units_required and urgent. Both entities also carry created_at and updated_at.

Data types. Textual attributes use CharField, the email attribute uses EmailField, units_required uses IntegerField, available and urgent use BooleanField, and the two timestamps use DateTimeField.

Relationships. No foreign key relationship exists between the two entities. A blood request is not bound to a particular donor, because any compatible donor may respond to it. The two are matched informally by comparing blood group and location values, so both tables stand alone in the schema.

CLIENT SIDE

11. Frontend Development

The entire interface is built with HTML5, CSS3 and vanilla JavaScript, served during development by Vite. There is a single HTML page; JavaScript shows and hides sections and renders lists into the DOM from the JSON returned by the API.

Home section

Introduces the application and provides navigation to the donor directory, the search page, the registration form and the blood requests section.

Donor Directory

Requests the full donor list from the API when the section is opened and renders each donor as a card showing name, blood group, location, phone and availability status.

Find a Donor

Provides a blood group dropdown and a location input. On submission the values are used to filter the donor list, and only matching donors are rendered.

Blood Requests

Lists blood requests with the requester name, blood group, location, phone, hospital, units required and urgency, with urgent requests visually distinguished.

Register Donor

A form collecting name, blood group, phone, email, location and availability. The values are validated and sent to the API as a POST request, after which the directory is refreshed.

Edit and Delete controls

Each donor card carries an Edit control, which loads the record into the form for a PUT request, and a Delete control, which issues a DELETE request after a confirmation prompt.

Search functionality

Search is performed against the donor records returned by the API, matching on blood group and on location, and re-rendering the list with the filtered results.

Responsive interface

CSS is written so that the layout adapts to smaller screens, with cards and form fields stacking vertically on mobile widths.

Client-side validation

Before any request is sent, JavaScript confirms that required fields are filled, that a blood group has been selected, and that the phone number and email are correctly formatted.

SCREENS

12. User Interface

The interface consists of the following screens, each corresponding to a section described in Chapter 11.

Screen	What it shows
Home Page	Introduction to RareBlood with navigation to all other sections.
Donor Directory	Every registered donor, showing name, blood group, location, phone and availability.
Find a Donor	Search form and the resulting donors filtered by blood group and location.
Blood Requests	List of blood requests with hospital, units required and urgency.
Register Donor	Validated form used to add a new donor to the directory.
Edit Donor	The same form pre-filled with an existing donor's values for updating.
Delete confirmation	A confirmation prompt shown before a donor record is removed.

SERVER SIDE

13. Backend Development

Django project

The Django project config holds the settings module, the root URL configuration and the WSGI entry point. Installed applications include the REST framework and the project's own donors application.

Django application

The donors application contains the models, serializers, ViewSets and URL routing for both entities, keeping all domain logic in one place.

Models

Two model classes, Donor and BloodRequest, define the fields, their data types and their constraints. The blood_group field uses a fixed choices list so that invalid values cannot be stored.

```
class Donor(models.Model):
    BLOOD_GROUPS = [
        ('A+', 'A+'), ('A-', 'A-'), ('B+', 'B+'), ('B-', 'B-'),
        ('AB+', 'AB+'), ('AB-', 'AB-'), ('O+', 'O+'), ('O-', 'O-'),
    ]
    name = models.CharField(max_length=100)
    blood_group = models.CharField(max_length=3, choices=BLOOD_GROUPS)
    phone = models.CharField(max_length=15)
    email = models.EmailField(blank=True, null=True)
    location = models.CharField(max_length=100)
    available = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"{self.name} ({self.blood_group})"
```

Serializers

ModelSerializer classes convert model instances into JSON for responses and validate incoming JSON before it is saved, returning field-level error messages when validation fails.

ViewSets

ModelViewSet classes provide the list, create, retrieve, update and destroy actions for both entities from a single class, so all CRUD operations are generated from one queryset and one serializer.

```
class DonorSerializer(serializers.ModelSerializer):
    class Meta:
        model = Donor
        fields = '__all__'

class DonorViewSet(viewsets.ModelViewSet):
    queryset = Donor.objects.all().order_by('-created_at')
    serializer_class = DonorSerializer
```

URL routing

A DRF DefaultRouter registers the two ViewSets, generating the list endpoints and the detail endpoints automatically. These routes are included in the project URL configuration under the `/api/` prefix.

Django ORM and SQLite integration

The ORM maps model classes to SQLite tables and model instances to rows. Queries written as Python expressions are translated into SQL by Django, and the schema itself is created and altered through migration files generated by `makemigrations` and applied by `migrate`.

Server-side validation

Serializer validation runs on every create and update request. Missing required fields, invalid blood groups and badly formatted email addresses are rejected with HTTP 400 and a structured error body, regardless of what the browser allowed through.

API

14. REST API Documentation

All endpoints accept and return JSON. The development base URL is `http://127.0.0.1:8000`.

Method	Endpoint	Description
GET	<code>/api/donors/</code>	Retrieve all donors
POST	<code>/api/donors/</code>	Create a donor
GET	<code>/api/donors/{id}/</code>	Retrieve one donor
PUT	<code>/api/donors/{id}/</code>	Update donor
DELETE	<code>/api/donors/{id}/</code>	Delete donor
GET	<code>/api/blood-requests/</code>	Retrieve blood requests
POST	<code>/api/blood-requests/</code>	Create blood request
GET	<code>/api/blood-requests/{id}/</code>	Retrieve one request
PUT	<code>/api/blood-requests/{id}/</code>	Update request
DELETE	<code>/api/blood-requests/{id}/</code>	Delete request

14.1 Sample request — create a donor

```
POST /api/donors/
Content-Type: application/json

{
  "name": "Swetha R",
  "blood_group": "O-",
  "phone": "9876543210",
  "email": "swetha@example.com",
  "location": "Chennai",
  "available": true
}
```

14.2 Sample response — HTTP 201 Created

```
{
  "id": 7,
  "name": "Swetha R",
  "blood_group": "O-",
  "phone": "9876543210",
  "email": "swetha@example.com",
  "location": "Chennai",
  "available": true,
  "created_at": "2026-01-14T10:24:11.482Z",
  "updated_at": "2026-01-14T10:24:11.482Z"
}
```

14.3 Sample error response — HTTP 400 Bad Request

```
{
  "blood_group": ["This field is required."],
  "phone": ["This field may not be blank."]
}
```

14.4 Status codes returned

Code	Meaning	When it occurs
200 OK	Success	A successful GET or PUT request.
201 Created	Record created	A successful POST request.
204 No Content	Record deleted	A successful DELETE request.
400 Bad Request	Validation failed	Required field missing or an invalid value submitted.
404 Not Found	Record not found	The requested id does not exist in the database.

IMPLEMENTATION

15. CRUD Implementation

Create

The Register Donor form collects the donor details. JavaScript validates the input, converts it to JSON and sends a POST request to `/api/donors/`. The serializer validates the payload on the server and the ORM inserts a new row into the donor table. A 201 response is returned, a success message is shown, and the directory is refreshed so the new donor appears immediately.

Read

When the directory or the blood requests section is opened, a GET request retrieves the records from the API. The ViewSet returns the queryset serialized as a JSON array, and JavaScript iterates over it to build the list in the DOM. A GET request to a detail endpoint retrieves a single record by its id.

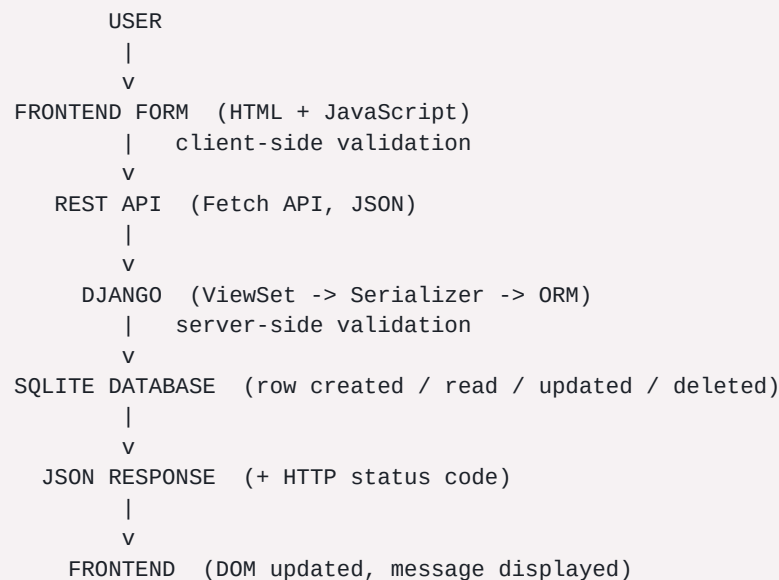
Update

Selecting Edit loads the donor's current values into the form. On submission a PUT request is sent to `/api/donors/{id}/` with the complete set of fields. The serializer revalidates the data and the ORM updates the existing row, refreshing the `updated_at` timestamp. This is most often used to change a donor's availability.

Delete

Selecting Delete shows a confirmation prompt. On confirmation a DELETE request is sent to `/api/donors/{id}/`. The ViewSet retrieves the instance by primary key and removes the row from SQLite, returning 204 No Content, after which the list is re-rendered.

CRUD workflow diagram



INTEGRATION

16. Frontend–Backend Integration

The two halves of the project are joined only by HTTP. The frontend holds no database logic and the backend produces no HTML; everything passes between them as JSON.

- **Fetch API** — all requests are issued from JavaScript using `fetch` with `async` and `await`, so the page never reloads while data is exchanged.
- **HTTP requests** — GET retrieves records, POST creates them, PUT updates them and DELETE removes them, each sent to the matching list or detail endpoint.
- **JSON** — outgoing data is serialized with `JSON.stringify` and the Content-Type header is set to `application/json`; incoming responses are parsed with `response.json()`.
- **API responses** — the response status is checked before the body is used, so validation failures and missing records are detected rather than silently ignored.
- **Dynamic UI updates** — after every successful operation the affected list is re-rendered from the API response, keeping the interface consistent with the database.
- **Success messages** — confirmation messages are displayed after a record is created, updated or deleted.
- **Error messages** — validation errors returned by the serializer are shown against the relevant fields, and connection failures produce a clear message.

Example: creating a donor from the frontend

```
async function registerDonor(donor) {
  try {
    const response = await fetch('http://127.0.0.1:8000/api/donors/', {
      method : 'POST',
      headers: { 'Content-Type': 'application/json' },
      body    : JSON.stringify(donor)
    });

    if (!response.ok) {
      const errors = await response.json();
      showError(errors);
      return;
    }

    await response.json();
    showSuccess('Donor registered successfully');
    loadDonors();
  } catch (error) {
    showError('Unable to reach the server. Please try again.');
```

DATA INTEGRITY

17. Validation & Error Handling

17.1 Validation applied

Rule	Client-side	Server-side
Required fields must not be empty	Checked before the request is sent	Enforced by model and serializer
Blood group must be a valid group	Restricted to a dropdown of eight values	Enforced by the choices list
Phone number format	Checked for digits and length	Enforced by field constraints
Email format, when supplied	Checked against a standard pattern	Validated by EmailField
Units required must be positive	Checked on the request form	Validated by the serializer

17.2 Error handling

- **Invalid request handling** — a 400 response is parsed and the returned field errors are shown beside the inputs that caused them.
- **Record not found** — a request for an id that no longer exists returns 404 and a clear message is displayed instead of an empty screen.
- **Backend connection errors** — if the Django server is not running, the fetch call throws, the error is caught, and the user is told that the server cannot be reached.
- **Empty results** — when a search returns no matches, an informative message is rendered rather than a blank list.
- **Accidental deletion** — a confirmation prompt is shown before any delete request is sent.

17.3 Client-side compared with server-side validation

	Client-side validation	Server-side validation
Runs in	The browser, before the request is sent	Django and DRF, before the record is saved
Purpose	Immediate feedback and fewer wasted requests	Authoritative protection of the stored data
Can be bypassed	Yes — by disabling JavaScript or calling the API directly	No — every request passes through it
Result on failure	Inline message; no request is sent	HTTP 400 with structured field errors

Both layers are therefore necessary. Client-side validation exists for the user's convenience, while server-side validation is what actually guarantees that no invalid record reaches the database.

QUALITY ASSURANCE

18. Testing

Testing was carried out manually through the browser interface and independently verified through the Django REST Framework browsable API and the Django admin, confirming in each case that the change had actually reached the database.

Test Case	Expected Result	Result
Create donor	Donor is created	Passed
Read donors	Donor list displayed	Passed
Update donor	Information updated	Passed
Delete donor	Donor removed	Passed
Search donor	Matching donors displayed	Passed
Invalid input	Validation message displayed	Passed
API connection	Data exchanged correctly	Passed

18.1 Result summary

All seven test cases passed. The four CRUD operations were verified end to end, search returned the expected filtered results, invalid input was rejected with a visible message, and data was confirmed to persist in SQLite after the server was restarted.

ENGINEERING PRACTICE

19. Security & Quality

The application does not implement user authentication, and no claim of authenticated or role-based access is made. The following practices were nevertheless applied to keep the project safe and maintainable.

- **No credentials in source code** — no passwords or API keys are stored in the repository.
- **Django ORM for all database access** — queries are written through the ORM rather than as raw SQL, so parameters are escaped automatically and SQL injection is avoided.
- **Input validation** — every create and update request is validated by the serializer before any record is written.
- **Separation of concerns** — the frontend, the backend and the database are separate layers communicating only through defined interfaces.
- **Meaningful naming** — models, fields, functions and files are named descriptively so the codebase can be read without additional explanation.
- **.gitignore** — the virtual environment, node_modules, cache files and local artefacts are excluded from version control.
- **Error handling** — failed requests are caught and reported instead of leaving the interface in an inconsistent state.

VERSION CONTROL

20. Git & GitHub

The project is maintained under Git from the beginning of development and hosted on GitHub.

- **Repository creation** — a Git repository was initialised in the project root with `git init`.
- **Commits** — work was committed in logical stages, such as adding the models, adding the API endpoints, building the frontend sections and adding validation.
- **GitHub repository** — the local repository was linked to a remote on GitHub and pushed, giving an off-machine backup and a visible commit history.
- **README** — the repository includes a README describing the project, the technology stack and the steps required to run it.
- **Source code management** — the frontend, backend and migrations are tracked together in a single repository.
- **.gitignore** — keeps the virtual environment, node_modules and generated files out of the commit history.

```
git init
git add .
git commit -m "Add donor model, serializer and API endpoints"
git branch -M main
git remote add origin https://github.com/swetha745/rare-blood-donor-directory.git
git push -u origin main
```

Repository: github.com/swetha745/rare-blood-donor-directory

GETTING STARTED

21. Installation & Execution

Two servers must be running at the same time, in two separate terminals: the Django backend and the Vite frontend.

Backend (first terminal)

```
cd C:\Users\swetha\rare-blood-donor-directory\backend

python manage.py migrate
python manage.py runserver
```

The backend is then available at `http://127.0.0.1:8000/` and the API at `http://127.0.0.1:8000/api/donors/`. Migrations only need to be applied again after the models are changed.

Frontend (second terminal)

```
cd C:\Users\swetha\rare-blood-donor-directory

npm run dev
```

The frontend is then available at `http://localhost:5173/`. Open this URL in the browser to use the application. If the Django server is not running, the interface will load but will display a connection error message in place of the donor data, because every list is fetched from the API.

First-time setup

On a machine where the project has not been run before, install the Python dependencies inside a virtual environment and the frontend dependencies with npm, before running the commands above.

```
# backend
python -m venv venv
venv\Scripts\activate
pip install django django-rest-framework django-cors-headers
python manage.py makemigrations
python manage.py migrate

# frontend (project root)
npm install
```

LESSONS LEARNED

22. Challenges & Solutions

Challenge	Solution
Connecting the frontend and the backend	Used the Fetch API to call REST endpoints and exchange JSON, keeping the two layers fully decoupled.
Database integration	Used the Django ORM with SQLite, so tables and queries are handled through model classes and migrations instead of raw SQL.
Implementing CRUD operations	Used Django REST Framework ModelViewSets, which generate list, create, retrieve, update and destroy actions from a single class.
Form validation	Added client-side checks in JavaScript for immediate feedback and serializer validation on the server as the authoritative check.
Searching donors	Added filtering by blood group and location so that only relevant donors are displayed.
API errors and failed requests	Checked the response status before using the body, and displayed serializer errors and connection failures as readable messages.
Version control	Used Git for staged commits and GitHub as the remote repository, with a .gitignore excluding the virtual environment and node_modules.

WHAT'S NEXT

23. Future Enhancements

#	Enhancement	Benefit
1	User authentication	Only the donor who created a record could edit or delete it.
2	Admin and user roles	Separate privileges for coordinators and ordinary users.
3	Email and SMS emergency notifications	Compatible donors would be alerted automatically when an urgent request is raised.
4	Location-based donor matching	Donors could be matched by coordinates rather than by city name alone.
5	Nearby donor search	Results could be ranked by distance from the hospital.
6	Mobile application	The same REST API could serve a mobile client without backend changes.
7	Advanced donor filtering	Filtering by availability, age group or last donation date.
8	Real-time blood request notifications	Requests could reach donors the moment they are raised.

OUTCOMES

24. Learning Outcomes

- **Full-stack development** — understood how a browser client, an application server and a database fit together, and how responsibility is divided between them.
- **HTML, CSS and JavaScript** — built a complete interface without a framework, including DOM rendering, event handling and responsive styling.
- **Django** — learned the project and application structure, models, migrations, URL routing and the admin interface.
- **REST APIs** — understood resources, HTTP methods, status codes, and how serializers and ViewSets expose models as JSON endpoints.
- **SQLite** — learned how a relational database stores records, and how primary keys and data types are defined.
- **CRUD operations** — implemented and traced create, read, update and delete from a form click all the way to a database row.
- **Database integration** — used the Django ORM to work with tables as Python objects instead of writing SQL.
- **Validation** — learned why validation must exist on the server as well as in the browser.
- **Git and GitHub** — practised staged commits, remote repositories and the use of .gitignore.
- **Testing and debugging** — learned to use browser developer tools, the network tab and the DRF browsable API to locate and fix problems.

SUMMARY

25. Conclusion

RareBlood provides a centralised system for registering, searching and managing blood donor information using a full-stack architecture. Donor records can be created, viewed, edited and deleted, the directory can be filtered by blood group and location, and blood requests can be recorded with the hospital, units required and urgency clearly indicated. All data is validated on both the client and the server and stored persistently in SQLite.

Every objective set out at the start of the project has been met. The four CRUD operations are implemented and tested for the donor entity, the search feature returns correctly filtered results, validation prevents invalid records from being stored, and the frontend and backend are fully integrated over a REST API.

The decoupled design is the project's main strength. Because the backend exposes a clean JSON API rather than rendering HTML, the same server could later support a mobile application or a redesigned interface without any change to the server-side code. Beyond the technical result, the project addresses a real problem: reducing the time taken to locate a compatible donor during an emergency, which matters most for exactly the rare blood groups the system was designed around.

SOURCES

26. References

#	Reference	URL
1	Django documentation	https://docs.djangoproject.com/
2	Django REST Framework documentation	https://www.django-rest-framework.org/
3	Python documentation	https://docs.python.org/3/
4	SQLite documentation	https://www.sqlite.org/docs.html
5	Vite documentation	https://vitejs.dev/guide/
6	Git documentation	https://git-scm.com/doc
7	GitHub documentation	https://docs.github.com/
8	MDN Web Docs — Fetch API	https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

RAREBLOOD

Rare Blood Donor Alert Directory

27. GitHub Repository

`github.com/swetha745/rare-blood-donor-directory`

The repository contains the backend Django project, the vanilla JavaScript frontend, the database migrations, the README and the project configuration files.

Thank You